

2026 “钉耙编程” 中国大学生算法设计 暑期联赛（2）题解

2026 年 7 月 23 日

概况

- 预期难度：
 - $HKJD < ECGAL < FBI$
- 实际通过人数排序：
 - $HKJ < DG < FABCE < LI$

现场情况

通过人数

1001	1002	1003	1004	1005	1006	1007	1008	1009	1010	1011	1012
51/248	37/99	29/159	327/1062	14/37	81/583	192/419	635/2325	0/8	508/1652	597/1304	0/193

首次通过

A	B	C	D	E	F
0:36	0:27	0:16	0:20	1:08	0:20
G	H	I	J	K	L
0:25	0:10		0:14	0:04	

最遥远的距离

题目大意

- 给出一个长度为 n 的序列 $d_1, d_2, \dots, d_n$ 。
- 你需要还原任意一棵树（或判断无解），使得对于每个点 i 到树中最远点的距离恰好等于 d_i 。定义两点间的距离为路径上的点数。
- $1 \leq n \leq 10^5, 1 \leq d_i \leq n$ 。

最遥远的距离

- 首先 d 数组的最大值等于该树最长链的长度，那么有个必要条件就是**这条最长链必须存在**。
- 具体地，设 L, R 分别为 d 数组的最小值与最大值，那么 $L = \lfloor \frac{R}{2} \rfloor + 1$ ，对于所有满足 $L < i \leq R$ 的 i 都要在 d 数组中至少出现两次，并且当 R 是奇数时 L 要恰好出现一次， R 是偶数时 L 要恰好出现两次。
- 发现该条件也是充分的，我们先构造出一条最长链，那么剩下还没加入树中的节点 x 必定满足 $L < d_x \leq R$ ，只需将 x 连向这条最长链的第 $R - d_x + 2$ 个点即可。
- By the way: 一个点在树上的最远点，必定为树的直径的两个端点的其中之一。这个经典结论或许会更好帮助你理解此题。

键盘杀手

题目大意

- 给出一个长度为 n 的序列 $a_1, \dots, a_n$ (特别地 $a_0 = a_{n+1} = 0$)。
- 你可以进行若干次操作, 每次操作形如: 选择一个正整数 i ($1 \leq i \leq n$), 花费 $\max(a_{i-1}, a_{i+1})$ 的代价将 a_i 赋值为 0。
- 求出将 $a_1, \dots, a_n$ 均赋值为 0 的最小代价总和。
- $1 \leq n \leq 10^5$, $1 \leq a_i \leq 10^9$ 。

键盘杀手

- Key Observation: 假设给这 n 个数定一个操作顺序, 那么删除 i 对代价总和的贡献, 只取决于 $i-1, i, i+1$ 的删除顺序。
- 考虑 dp, 设 $f_{i,0/1}$ 分别表示已经考虑了操作 $1, \dots, i$ 的代价, 并且 i 晚于/早于 $i-1$ 操作时的最小代价总和。转移方程比较简单。
- 时间复杂度 $\mathcal{O}(n)$ 。

幻灵战队 2

题目大意

- 给出一个长度为 n 的胜负序列（其中 0 表示失败，1 表示胜利）。
- 初始时科技点为 0，连续失败的第 x 场对战将会获得 $20 + 5x$ 的科技点。
- 你可以将至多 k 场失败改成胜利，使得重新计算后的科技点总和最小。求出可能获得的科技点总和的最小值。
- $1 \leq n \leq 10^5$, $0 \leq k \leq n$ 。

幻灵战队 2

- 首先，连败获得科技点的式子 $20 + 5x$ 的系数，不影响求解答案，所以不妨认为式子是 x 。
- 记 $f(i, j)$ 表示：将一个长度为 i 的全 0 段的 j 个 0 改成 1 之后，科技点总和的最小值。
- 相当于将 $i - j$ 个数划分到 $j + 1$ 个分段里，一个长度为 L 的分段代价为 $\frac{L(L+1)}{2}$ ，要求所有段的代价之和最小。
- 最优的策略一定是平均分**，因为如果有两个段段长相差 > 1 ，那么将段长之差缩小一定会使得代价变小。那么有

$$f(i, j) = \frac{u(u+1)}{2} \cdot (j+1-r) + \frac{(u+1)(u+2)}{2} \cdot r$$

其中 $u = \lfloor \frac{i-j}{j+1} \rfloor$, $r = (i-j) \bmod (j+1)$ 。

幻灵战队 2

- 回到原问题，需要先找出所有的全 0 段，显然不同的全 0 段之间是相互独立的。我们要考虑的是如何分配操作次数。
- 发现当 i 固定时， $f(i, j) - f(i, j + 1)$ 是随着 j 的增大而减小的。也就是说，**每一段的“更新”收益是单调不增的。**
- 所以，每次取出当前更新收益最大的段，令其操作次数 $+1$ 一定是最优的。可以使用大根堆维护上述过程。
- 时间复杂度 $\mathcal{O}((n + k) \log n)$ 。

坪厠鸡

题目大意

- 一场比赛有 n 支队伍，以及 k 台完全相同的评测机。比赛共 m 次提交，第 i 次提交由队伍 a_i 在第 b_i 秒发起，需要消耗 c_i 秒进行评测。**保证所有的 b_i 严格单调递增。**
- 每台评测机同一时刻至多只能处理一份提交。
- **每支队伍同一时刻至多只能有一份提交正在评测。**
- 每次调度时，会从所有等待提交中，选择提交时间 b_i 最早的一份合法提交开始评测。直到没有空闲评测机或没有可评测提交。
- 若同时有提交到达和评测结束，则先处理完所有事件再进行调度。
- 求每个提交实际开始评测的时间。
- $1 \leq n, m, k \leq 2 \times 10^5$ 。

坪厠鸡-解法 1

- 维护两个小根堆：
 - running 堆：已经分配到评测机的提交，按评测结束时间排序。
 - pending 堆：所有当前无正在评测提交的队伍，其尚未处理的最早提交，按提交时间排序。
- 初始时，running 储存着 k 份结束时间为 0 且不属于任何队伍的提交（用来模拟评测机），pending 储存着每个队伍的第一次提交。
- 重复以下步骤，直到每一个提交的评测时间都被确定：
 - 弹出 running 的堆顶，相当于释放一台评测结束时间最早的评测机，设这份提交的评测结束时间为 t ，属于队伍 a 。若队伍 a 后续仍有提交，则将队伍 a 的下一份提交插入 pending 之中。
 - 若 pending 非空，则弹出 pending 的堆顶，设这份提交的编号为 i ，则我们**直接将第 i 次提交分配给刚释放的评测机，并确定评测开始时间为 $\max(t, b_i)$** 。请读者思考这样做的正确性。
- 注意初始时要令 $k \leftarrow \min(k, n)$ ，因为本题没有限制 $\sum k$ 的上界。

坪厠鸡-解法 2

- 其实只有两种时间是我们需要关心的：有队伍提交代码的时间，以及有提交结束评测的时间。
- 使用小根堆维护这些要处理的时间（暂且称为 time 堆），同时还需要维护：
 - running 堆：正在评测的提交，按评测结束时间排序。
 - pending 堆：有空闲评测机就可以评测的提交，按提交时间排序。
 - 每支队伍的等待队列。
 - 当前空闲评测机数量。
- 重复以下步骤，直到每一个提交的评测时间都被确定：
 - 在 time 中取出最早的时间 t 。
 - 处理 running 中所有在 t 时间结束评测的提交，处理在 t 时间到达的新提交。
 - 检查在 t 时间变为空闲或有新提交到达的队伍 a ，若 a 既不在 running 也不在 pending，就将其等待队列的队首加入 pending。
 - 若当前有空闲评测机，不断取出 pending 的堆顶开始评测。

减数游戏 2

题目大意

- 「减数游戏」在一个长度为 n 的正整数序列 $a_1, \dots, a_n$ ($1 \leq a_i \leq n$) 上进行, 这些数构成了一个可重集合 S .
 - 两人轮流操作, 河灵先手。
 - 每次选择一个正整数 x ($1 \leq x \leq \min\{S\}$), 将集合中所有数减去 x , 并删除变成 0 的数。
 - 若本次操作删除了至少一个数, 则对方得 1 分。
 - 集合为空时游戏结束, 若河灵最终得分不少于胖胖龙, 则河灵获胜。
- 现给出一个长度为 n 的残缺序列 $a_1, \dots, a_n$ ($0 \leq a_i \leq n$), 其中 $a_i = 0$ 表示未知数, 可以独立替换成 $1 \sim n$ 中的任意正整数。
- 求有多少种替换方案, 使得双方均采取最优策略时河灵能够获胜。答案对 998244353 取模。
- $1 < n < 10^5$ 。

减数游戏 2

- 先考虑博弈获胜的条件是什么。首先可以对 S 中的元素进行去重，设去重后的集合从小到大的数分别为 $e_1, \dots, e_k$ ，显然我们只关心这些数的差分数组 $d_i = e_i - e_{i-1}$ 。
- 当 $d_i = 1$ 时，相当于强制让对方得一分，且下回合对方先手。
- 当 $d_i \geq 2$ 时，**当前玩家就可以掌握主动权**。设它后面紧接着 c 个连续的 1，那么：
 - 若 c 为奇数，则当前玩家选择 $x = d_i$ 。到达下一个 $d_j \geq 2$ 时，仍然是当前玩家操作，并且这一段双方得分相同。
 - 若 c 为偶数，则当前玩家选择 $x = d_i - 1$ 。到达下一个 $d_j \geq 2$ 时，仍然是当前玩家操作，并且这一段净胜一分。
- 因此一旦遇到 $d_i \geq 2$ 时，当前玩家都可以保持下一个 $d_j \geq 2$ 位置的先手权，并且分数差不减。
- 所以我们只关心开头有多少个连续的 1，博弈获胜的条件即为 $\text{mex}^+(S)$ **为奇数**，其中 $\text{mex}^+(S)$ 表示不在 S 中的最小**正整数**。

减数游戏 2

- 考虑计数。设一共有 b 个未知位置，枚举 $\text{mex}^+(S)$ 的值 p ，显然 p 不能在已知数中出现，设 $1 \sim p-1$ 中有 r 个未出现的数，我们要让这 r 个数都至少出现一次。考虑容斥，钦定有 i 个数仍未出现

$$\begin{aligned} & \sum_{i=0}^r (-1)^i \binom{r}{i} (n-1-i)^b \\ & \sum_{i=0}^r (-1)^i \frac{r!}{i!(r-i)!} (n-1-i)^b \\ & r! \sum_{i=0}^r \frac{(-1)^i}{i!} (n-1-i)^b \frac{1}{(r-i)!} \end{aligned}$$

- 这是一个与 r 有关的卷积式，使用 NTT 优化即可。注意特判 $\text{mex}^+(S) = n+1$ 的情况。
- 时间复杂度 $\mathcal{O}(n \log n)$ 。

张力

题目大意

- 给出一个长度为 n 非负整数序列 $a_1, \dots, a_n$ ，你需要将这些数重新排列成 $b_1, \dots, b_n$ ，最小化：

$$\sum_{i=1}^{n-1} \text{lowbit}(b_i \oplus b_{i+1})$$

- $1 \leq n \leq 5 \times 10^3, 0 \leq a_i < 2^{50}$ 。

张力

- $\text{lowbit}(x \oplus y)$ 取决于 x, y 从低到高的第一个不同的二进制位。
- 于是对 a_i 按照二进制位从低到高建立 01 Trie, $\text{lowbit}(x \oplus y)$ 的值和 x, y 在 Trie 上的 LCA 深度有关。
- 设 $f(u, i)$ 表示子树 u 中的所有元素排列成 i 个极长的**连续段**时的最小代价。同一子树内的连续段不能直接拼接，只能与其他子树的连续段交替排列，间接地归入同一个连续段。记点 u 在 Trie 上的深度为 h ，则将左右子树的两个连续段拼接起来的代价为 2^h 。
- 假设点 u 的左右儿子 L, R 中分别有 i, j 个连续段，将这些连续段重新排列成 k 个连续段（只有来自不同子树的连续段可以拼接），可行 k 的取值范围是 $\max(1, |i - j|) \leq k \leq i + j$ ，有转移方程

$$f(u, k) = \min_{|i-j| \leq k \leq i+j} \{f(L, i) + f(R, j) + (i + j - k) \times 2^h\}$$

- 直接枚举转移是 $\mathcal{O}(n^3)$ 的。

张力

- 改成枚举 k, i , 此时 j 的取值范围为

$$\max(1, |k - i|) \leq j \leq \min(\text{size}_R, k + i)$$

- 不妨设 $\text{size}_L \leq \text{size}_R$ (否则交换 L, R), 枚举较小子树的 i , 使用 ST 表维护较大子树中 $f(R, j) + j \times 2^h$ 的区间最小值。
- 使用 `std::vector` 维护 dp, 由于每个叶子会贡献树高 $\log A$ 个 dp 状态, 所以 dp 数组总长度不超过 $\mathcal{O}(n \log A)$ 。
- 时间复杂度 $\mathcal{O}(n^2 + n \log n \log A)$ 。
- Bonus: 尝试做到时间复杂度 $\mathcal{O}(n \log A)$ 。

另一个 shu 论问题

题目大意

- 给出一棵包含 n 个点的树，根节点为 1。
- 求出有多少对 x, y ($1 \leq x < y \leq n$) 满足 $\gcd(x, y) = \text{lca}(x, y)$ 。
- $1 \leq n \leq 2 \times 10^5$ 。

另一个 shu 论问题

- 记 $\gcd(x, y) = \text{lca}(x, y) = u$, 在 u 处统计答案。
- 我们称满足 $\text{lca}(x, y) = u$ 的点 (x, y) 为“好点对”，记：
 - $f(d)$ 表示满足 $\gcd(x, y) = d$ 的好点对个数。
 - $g(d)$ 表示满足 x, y 是 d 的倍数的好点对个数。

$$g(u) = \sum_{u|d} f(d)$$

- 由莫比乌斯反演，得

$$f(u) = \sum_{u|d} g(d) \mu\left(\frac{d}{u}\right)$$

- 所以相当于是要求 $\sum_{u|d} g(d) \mu\left(\frac{d}{u}\right)$ 。

另一个 shu 论问题—解法 1: dsu on tree

- 使用 dsu on tree, 维护子树内有多少个 d 的倍数 (记为 $cnt(d)$)。
- 每次加入一个轻子树时, 先统计答案, 再更新数组。
- 每当新加入点 x , 枚举 x 的所有约数 d , 此时 $g(d)$ 增加了 $cnt(d)$, 当 $u \mid d$ 时, 答案增加了 $cnt(d)\mu\left(\frac{d}{u}\right)$ 。
- 由于 $1 \sim n$ 中所有数的约数个数之和为 $\mathcal{O}(n \log n)$, 可以理解为对一棵包含 $\mathcal{O}(n \log n)$ 个点的树进行 dsu on tree (所以此时重儿子的定义为子树内约数个数之和最多的儿子)。
- 时间复杂度 $\mathcal{O}(n \log^2 n)$ 。

另一个 shu 论问题—解法 2：虚树

- 对于每个 d ，考虑统计 $g(d)$ 对答案的贡献。
- 将所有 d 的倍数作为关键点，建一棵虚树。只有虚树上的点才有可能作为关键点的 lca。
- 遍历虚树，求出虚树上每个点 u 对应的 $g(d)$ ，即有多少个关键点点对的 lca 为 u 。当 $u \mid d$ 时，对答案的贡献为 $g(d)\mu\left(\frac{d}{u}\right)$ 。
- 按照 dfs 序依次遍历所有点 x ，枚举 x 的所有约数 d ，将 x 加入 d 的关键点集合。这样所有关键点集合都按照 dfs 序排序，即可省略建虚树时的排序。配合欧拉序 + ST 表求 lca，即可线性建虚树。
- 时间复杂度 $\mathcal{O}(n \log n)$ 。

xyz 问题

题目大意

- 有 n 个布尔变量 $x_i \in \{0, 1\}$ 以及 m 个运算符变量 $op_j \in \{\&, |, \wedge\}$ 。
- 有 k 个条件，每个条件都包含四个整数 i, j, y, z ，表示

$$x_i \ op_j \ y = z$$

其中 $y, z \in \{0, 1\}$ 。

- 请还原出任意一组满足所有条件的 x_i 和 op_j ，或判断无解。
- $1 \leq n, m, k \leq 3 \times 10^5$ 。

xyz 问题

- 本题可以先把每个运算符变量的候选集合压缩到**最多两种**，从而转化为 2-SAT 问题。
- 对于每个 op_j ，观察其出现过的条件 (y, z) 类型，重点考虑 $(1, 0)$ 是否出现过：

1 **出现过** $(1, 0)$ ：即 $x_i \text{ } op_j \text{ } 1 = 0$ ，此时 $\mid$ 不可能满足条件，于是

$$op_j \in \{\&, \wedge\}$$

2 **没出现过** $(1, 0)$ ：相当于 op_j 只出现 $(0, 0)$, $(0, 1)$ 和 $(1, 1)$ 类型，此时 $\mid$ 一定优于 $\wedge$ ，所有选择 $\wedge$ 的可行方案都可以替换成选择 $\mid$ ，具体地

- $(0, 0)$, $(0, 1)$ 类型： $\mid$ 和 $\wedge$ 对 x_i 的要求相同。
- $(1, 1)$ 类型： $\mid$ 恒成立。

$$op_j \in \{\&, \mid\}$$

xyz 问题

- 综上，每个运算符变量最终至多只有两个候选值，使用一个布尔变量表示其选择第一个候选还是第二个候选。
- 对于每个条件，枚举 x_i 和 op_j 的所有取值，检查哪些组合不满足条件。每个不合法的组合，都可以转换成 2-SAT 中的限制条件。
- 使用 SCC 求解 2-SAT，即可还原出一组解（或判断无解）。

最后一次相遇

题目大意

- 数轴上有 n 个小球，第 i 个小球的初始坐标为 x_i 速度为 v_i ，其在时刻 t ($t \geq 0$) 的位置为 $x_i + v_i \cdot t$ 。
- 两个小球在某个时刻 $t \geq 0$ 处于同一位置时，称它们相遇，相遇后仍保持原速度运动。
- 有 Q 次询问，每次询问给出 l, r, a, b ，表示加入一个初始坐标为 a 速度为 b 的临时小球。**求临时小球与编号在 $[l, r]$ 内的小球相遇的所有时刻中的最大值。**若均不相遇，则输出 -1 。
- 保证不会出现两个初始坐标与速度均相同的小球，所以任意两个小球至多只会相遇一次。
- $1 \leq n, Q \leq 10^5$, $-10^9 \leq x_i, v_i, a, b \leq 10^9$ 。

最后一次相遇

- 小球 i 与临时小球的相遇时间 t 满足

$$x_i + v_i \cdot t = a + b \cdot t$$

$$t = -\frac{a - x_i}{b - v_i}$$

- 将一个初始坐标为 x 速度为 v 的小球视为二维平面上的点 (v, x) , 那么两个小球的相遇时间即为**两点之间斜率的相反数**。
- 所以等价于求询问点与区间 $[l, r]$ 内所有点斜率的最小值, 然后再取其相反数。虽然要求相遇时刻 $t \geq 0$, 但如果算出的答案小于 0, 也就相当于均不相遇。

最后一次相遇

- 对于询问点 (x, y) ，将查询点集按照横坐标与 x 的大小关系分为左右两侧，则答案点一定在左侧的上凸壳或者右侧的下凸壳。
- 将询问的区间拆成线段树上的 $\mathcal{O}(\log n)$ 个区间，然后将这次询问挂在线段树的这些对应节点上。
- 对于线段树上的区间 $[l, r]$ ，将区间 $[l, r]$ 内的所有点和询问点放在一起按照横坐标从小到大排序，从左到右扫描询问点，动态维护询问点左侧点集的上凸壳，然后在该凸壳上二分切线即可。右侧点集的贡献同理。
- 时间复杂度 $\mathcal{O}(n \log^2 n)$ 。

合成大 hdu

题目大意

- 给出一个正整数 n ($1 \leq n \leq 10^9$)。
- 构造一个仅包含 h, d, u 的字符串 S ($1 \leq |S| \leq 3001$), 使得字符串 S 的不同子序列 hdu 的个数恰好等于 n 。

合成大 hdu

- 构造子序列 hdu 个数为 n 的字符串，字符串长度至少为 $3\sqrt[3]{n}$ 。
- 所以本题的上界比较接近理论下界，最终构造出的字符串，字符 h, d, u 的数量都应该接近 1000。
- 先考虑怎么 check，也就是计算字符串的子序列 hdu 个数。可以从左到右使用增量法，设 a, b, c 表示当前字符串的子序列 h, hd, hdu 个数。那么在当前字符串的末尾添加一个字符的影响为：
 - 添加 h: $a \leftarrow a + 1$ 。
 - 添加 d: $b \leftarrow b + a$ 。
 - 添加 u: $c \leftarrow c + b$ 。
- 最终要使得 c 变成 n 。

合成大 hdu: $n \leq 1500^2$

- 特判一下 n 较小的情况。
- 当 $n \leq 1500^2$ 时, 令 $n = 1500q + r$ 。
- 若 $r = 0$, 构造方案为

$$h^{1500} d^q u$$

- 若 $r > 0$, 构造方案为

$$h^r d h^{1500-r} d^q u$$

合成大 hdu: $n > 1500^2$

- 当 $n > 1500^2$ 时, 由于三类字符的数量都接近 1000, 尝试用 1000 为基底构造字符串。
- 先构造前缀 $h^r d h^{1000-r}$, 然后在后面放 q 个 d , 此时添加 u 产生的子序列数量为

$$1000q + r$$

再在后面放一个 d , 此时添加 u 产生的子序列数量为

$$1000(q + 1) + r$$

- 此时我们有两个相近的贡献值, 让其中一部分 u 使用贡献值 1, 另一部分 u 使用贡献值 2:

$$h^r d h^{1000-r} d^q u^{999-s} d u^s$$

- 此时子序列 hdu 的数量为 $1000(999q + s) + 999r$ 。

合成大 hdu: $n > 1500^2$

- 考虑将 n 分解成 $1000(999q + s) + 999r$ 的形式。
- 选择合适的 r , 使得 $n - 999r$ 是 1000 的倍数, 注意到

$$n - 999r \equiv n + r \pmod{1000}$$

于是取 $r = 1000 - n \bmod 1000$ 即可。

- 然后记 $m = \frac{n-999r}{1000}$, 令 $m = 999q + s$, 三个参数 r, q, s 都已确认。
- 可以控制 $r \in [1, 1000]$, $q \in [0, 1000]$, $s \in [0, 999]$, 所以字符串的总长度不超过 3001。

表达式 2

题目大意

- 给出一个由 n 个非零数字构成的数字串 S 。
- 对每个 k ($0 \leq k < n$)，求出**插入恰好 k 个乘号**时，所有不同插入方案所得表达式结果之和。
- 答案对 998244353 取模。
- $1 \leq n \leq 10^5$ 。

表达式 2

- 先考虑朴素 dp。设 $f(i, k)$ 表示前 i 个数字插入了 k 个乘号，所有不同表达式结果之和；设 $g(i, k)$ 表示前 i 个数字插入了 k 个乘号，所有不同表达式除去最后一段后，其余各段乘积之和。
- 根据是否在 $i-1, i$ 之间插入乘号，有两种转移：
 - 不插入乘号：

$$f(i, k) \leftarrow_+ f(i-1, k) \times 10 + g(i-1, k) \times S_i$$

$$g(i, k) \leftarrow_+ g(i-1, k)$$

- 插入乘号：

$$f(i, k) \leftarrow_+ f(i-1, k-1) \times S_i$$

$$g(i, k) \leftarrow_+ f(i-1, k-1)$$

表达式 2

- 再考虑加速转移。首先将 dp 数组写成生成函数的形式。记

$$F_i(x) = \sum_{0 \leq k < i} f(i, k)x^k, \quad G_i(x) = \sum_{0 \leq k < i} g(i, k)x^k$$

- 可以认为 F_i 是一个 $i - 1$ 次的多项式, 这个多项式的 k 次项系数即为 $f(i, k)$ 。 G_i 同理。
- 然后将转移写成矩阵的形式:

$$[F_i \quad G_i] = [F_{i-1} \quad G_{i-1}] \times \begin{bmatrix} S_i x + 10 & x \\ S_i & 1 \end{bmatrix}$$

表达式 2

- 记 M_i 为从阶段 $i - 1$ 转移到阶段 i 的转移矩阵，那么相当于要求

$$[F_1 \quad G_1] \times M_2 \times M_3 \times \cdots \times M_n$$

- 使用分治求出 $M_2 M_3 \cdots M_n$ 即可，因为 $M_l \cdots M_r$ 中的多项式长度都是区间长度 $r - l + 1$ 级别的。
- 时间复杂度 $\mathcal{O}(n \log^2 n)$ 。

引力场

题目大意

- 给出一棵包含 n 个点的树，包裹初始位于节点 s ，目标是最终停在节点 t 。
- 当激活节点 x 时，若包裹当前位于与 x 直接相邻的节点，则包裹会被移动到 x ；否则包裹位置不变。
- 在 $n!$ 种不同的节点激活顺序中，有多少种节点激活顺序 $p_1, \dots, p_n$ 可以使得初始在节点 s 的包裹，在所有节点都激活完毕之后恰好停留在节点 t 。
- 答案对 998244353 取模。
- $1 \leq n \leq 10^5$ 。

引力场

- 首先讨论一下 $s = t$ 的情况，包裹的移动路径一定是 $s \rightarrow v \rightarrow s$ ，其中 v 是 s 的某个邻居。有如下的限制条件：
 - v 必须在 s 前激活。
 - s 的其他邻居的激活时间必须在 v, s 之间。
 - v 的其他邻居的激活时间不能在 v, s 之间。
- 先摆好 v, s ，再插入 s 的其他邻居，再插入 v 的其他邻居，最后再插入剩余的无关节点。该种情况对答案的贡献为

$$n! \cdot \frac{(\deg(s) - 1)! \cdot \deg(v)!}{(\deg(s) + \deg(v))!}$$

- 再讨论 $s \neq t$ 的情况，设树上从 s 到 t 的节点依次为 $p_1, \dots, p_m$ (其中 $p_1 = s, p_m = t$)，那么包裹的移动路径有两种情况：
 - 直接从 s 走向 t ：

$$s \rightarrow p_2 \rightarrow \dots \rightarrow p_{m-1} \rightarrow t$$

- 先走到 s 的某个邻居 a ($a \neq p_2$)，再走回 s ，再走向 t

$$s \rightarrow a \rightarrow s \rightarrow p_2 \rightarrow \dots \rightarrow p_{m-1} \rightarrow t$$

引力场

- 先考虑第一种情况，需要依次激活 $p_2, \dots, p_m$ 。于是先将 $p_2, \dots, p_m$ 摆好。
- 对于路径上相邻的点 p_i, p_{i+1} ，显然 p_i 的其他邻居的激活时间不能在 p_i, p_{i+1} 之间（下面称 p_i 的其他邻居不能在 p_i, p_{i+1} 的**空隙**内）。
- 特别地， s 的其他邻居不能在 p_2 之前， t 的其他邻居不能在 p_m 之后。这里各产生了一个空隙。
- 至于如何计数？考虑容斥。对于某个空隙，假设有 r 个点不应该出现在该空隙内，我们钦定其中 i 个节点必须出现在该空隙内。方案数为

$$\binom{r}{i} i! = \frac{r!}{(r-i)!}$$

引力场

- 记多项式

$$F_r(x) = \sum_{i=0}^r \frac{r!}{(r-i)!} x^i$$

该多项式的 i 次项系数，即为钦定 i 个节点必须出现在空隙内的方案数。

- 至于多个空隙的限制，可以将这些空隙对应的多项式乘起来。最终得到的多项式的 i 次项系数，即为**钦定 i 个节点非法**的方案数 h_i 。
- 假设我们已经摆好了 k 个数（在第一种情况下这些数为 $p_2, \dots, p_m$ ，此时 $k = m - 1$ ），在钦定 i 个节点非法的时候，先将这 i 个节点放到钦定的空隙里，再插入剩下的无关节点。所以对答案的贡献为

$$\sum_i (-1)^i \frac{n!}{(k+i)!} h_i$$

引力场

- 再考虑第二种情况，需要依次激活 $a, p_1, p_2, \dots, p_m$ ，于是先将 $a, p_1, p_2, \dots, p_m$ 摆好。
- 空隙的分析方法同第一种情况。但有一类节点较为特殊：注意到 s 的其他邻居既不能出现在 a 的前面，也不能出现在 p_1, p_2 之间。所以**这些节点不能出现在这两个空隙中**。
- 所以在钦定的时候，我们需要将其中的一部分划分到空隙 1 中，另一部分划分到空隙 2 中。所以这两个空隙所对应的多项式为

$$G_r(x) = \sum_{i=0}^r \frac{r!}{(r-i)!} (i+1)x^i$$

- 计算贡献的方式同第一种情况。
- 在计算贡献时，涉及到计算若干个次数之和 $\leq n$ 的多项式之积。可以每次取出两个次数最小的多项式相乘后放回。
- 时间复杂度 $\mathcal{O}(n \log^2 n)$ 。

Thank you!